NRV.NET — TABLEAU DE BORD DU PROJET

SAE Atelier-Projet de développement web 1 · IUT Nancy-Charlemagne · BUT 3 DWM · 19-23 octobre 2026

Groupe : Romain Santilli, Cinar Erdal, Eliot Schmitt

LIENS

Élément	Lien
Dépôt Git	https://github.com/Dev9Shadow/NRV-Atelier
Installation	`README.md`
Application (docketu)	`http://docketu.iutnc.univ-lorraine.fr:55001/`
Données et comptes de test	`docs/donnees-test.md`
Modèle du domaine	`docs/modele-domaine.png`

FONCTIONNALITÉS RÉALISÉES

Programme du festival

#	Fonctionnalité	État	Route(s) API	Page(s)
1	Liste des spectacles (titre, date, horaire, image)	Réalisée	`GET /spectacles`	`index.html`
2	Détail d'une soirée (nom, thématique, date, lieu, tarifs, spectacles)	Réalisée	`GET /soirees/{id}`	`soiree.html`
3	Clic sur un spectacle → détail de sa soirée	Réalisée	lien HATEOAS `links.soiree`	`index.html` → `soiree.html`
4	Filtrage par date	Réalisée	`GET /spectacles?date=`	`index.html`
5	Filtrage par style de musique	Réalisée	`GET /spectacles?style=`, `GET /styles`	`index.html`

Achat de billets

#	Fonctionnalité	État	Route(s) API	Page(s)
—	Inscription et connexion (JWT)	Réalisée	`POST /auth/register`, `POST /auth/signin`	`connexion.html`
6	Ajout de billets au panier (simple ou pass 3 jours, tarif normal ou réduit)	Réalisée	`POST /panier/lignes`, `GET /soirees`	`soiree.html`
7	Panier courant et montant total	Réalisée	`GET /panier`	`panier.html`
8	Validation du panier en commande	Réalisée	`POST /panier/validation`	`panier.html`

#	Fonctionnalité	État	Route(s) API	Page(s)
9	Paiement simulé, contrôle et mise à jour des places en transaction	Réalisée	`POST /commandes/{id}/paiement`, `GET /commandes/{id}`	`paiement.html`
10	Billets imprimables (acheteur, référence, date, catégorie), accès par token	Réalisée	`GET /billets/{token}`	`billet.html` (`@media print`)

Back-office

#	Fonctionnalité	État	Route(s) API	Page(s)
11	Jauge : places vendues et recette par soirée	Réalisée	`GET /admin/soirees` (rôle ADMIN)	`admin.html`

Fonctionnalités étendues

#	Fonctionnalité	État
1	Pagination de la liste des spectacles	Non réalisée
2	Modification du panier (quantités, suppression d'une ligne)	Réalisée (`PATCH` et `DELETE /panier/lignes/{id}`)
3	Back-office : ajout de spectacles et de soirées	Non réalisée
4	Vérification des places disponibles à l'ajout au panier	Réalisée (à l'ajout, à la modification et à la validation)

QUALITÉ

- Architecture hexagonale : actions Slim (adaptateurs d'entrée), use cases et entités du domaine, ports (interfaces), repositories PDO (adaptateurs de sortie), DTO
- Injection de dépendances avec PHP-DI (`api/config/dependencies.php`), middlewares CORS, authentification JWT, autorisation administrateur, gestionnaire d'erreurs JSON
- Sécurité : requêtes préparées uniquement, mots de passe hachés, validation des entrées dans les actions et le domaine, identifiant de l'utilisateur toujours lu dans le token
- Tests unitaires PHPUnit : 10 classes de tests sur le domaine et les use cases (`composer test`)
- Front : SCSS (variables, mixins `bp()`, `conteneur`, `grille-adaptative`, `focus-visible`), grid et flex, mobile-first, templates Handlebars, `fetch` sur l'API

CONTRIBUTIONS

Membre	Rôle	Réalisations principales
Romain Santilli	Infrastructure, comptes, back-office	Docker Compose, configuration Slim et PHP-DI, middlewares CORS et JWT, gestionnaire d'erreurs JSON, `ServiceCatalogue` et routes V1, authentification (`ServiceAuth`, repository utilisateur, provider JWT), jauge du back-office (use case, repository, action, page admin), page de connexion

Membre	Rôle	Réalisations principales
Cinar Erdal	Données et métier	Schéma SQL, seed réaliste (faker), entités du domaine et repositories PDO, tests du catalogue et du tarif, diagramme du modèle, domaine de la commande (`Commande`, `LigneCommande`, `Billet`) et ses tests, paiement en transaction (`ServicePaiement`, `CarteBancaire`) et ses tests, route et page du billet imprimable, page de paiement, données de test
Eliot Schmitt	Front et panier	Maquettes, scénario d'usage, base SCSS et composants, liste des spectacles, filtres, détail de soirée, client API (`api.js` : session, token, erreurs), réservation, page panier, `ServicePanier` et ses tests, routes `/panier` et `GET /soirees`, header commun, responsive des pages connexion et admin, README

Organisation : plan de travail partagé avec des contrats d'API fixés à l'avance, un commit par tâche, intégration régulière sur `main`.